

# Izrada agenta s generativnom umjetnom inteligencijom teksta specifične namjene

Diplomski rad

Izradio:  
Zlatko Pračić

Mentor:  
prof. dr. sc. Markus Schatten

12. rujna 2026.

- 1 Uvod
- 2 Zajednička implementacijska osnova
- 3 Tri konfiguracije chatbota
- 4 Usporedba pristupa
- 5 Statistička analiza rezultata
- 6 Iterativni razvoj – kako su nastale tri verzije
- 7 Zaključak
- 8 Bibliografija

# Uvod

## Cilj rada i tri konfiguracije

- Domena: **hrvatsko vojno zakonodavstvo** (4 dokumenta, 538 članaka, 1.414 stavaka). Ustav RH, Zakon o obrani, Zakon o službi u OS RH i Pravilnik o temeljnom vojnom osposobljavanju
- **Cilj:** izraditi i usporediti tri konfiguracije chatbota koje se razlikuju isključivo u načinu dohvata konteksta

### Vanilla

bez dohvata konteksta, oslanja se samo na predznanje modela

### RAG

dohvat iz vektorske baze (FAISS)

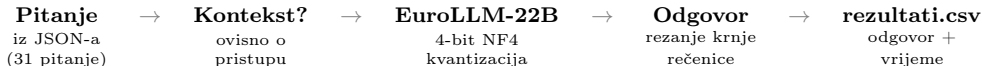
### GraphRAG

dohvat temeljen na grafu pravnih dokumenata (Neo4j)

**Evaluacija:** Tri verzije odgovora na 31 pitanje. Automatske metrike (ROUGE-L, BERTScore, kosinusna sličnost) + 13 ljudskih ocjenjivača + LLM-as-a-Judge (Claude, Gemini).

# Zajednička osnova

1/3 — Ista okosnica u sve tri bilježnice



## Identično u sve tri verzije:

- Isti LLM (**EuroLLM-22B-Instruct**), 4-bit NF4 kvantizacija (QLoRA [1]), pipeline "text-generation"
- Isti SYSTEM\_PROMPT, pravni stručnjaka za zakonodavstvo RH
- Ista generate\_response: 512 tokena, temperatura 0.3, repetition\_penalty 1.1
- Isti testni skup (31 pitanje) i isti rezultati.csv. Svaki pristup dodaje svoje stupce

Razlikuje se samo drugi korak – **dohvat konteksta**

# Zajednička osnova

## 2/3 – Učitavanje modela i kvantizacija

```
quantization_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_use_double_quant=True,
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_quant_type="nf4")

pipe = pipeline(
    "text-generation",
    model=model_name,           # EuroLLM-22B
    dtype=torch.float16,
    device_map="auto",
    model_kwargs={"quantization_config":
        quantization_config})

pipe.model.generation_config.max_length = 8192
```

- 4-bitna kvantizacija komprimira model od 22 mlrd. parametara da stane u memoriju GPU-a
- pipeline učitava komprimirani model spreman za generiranje i sam ga raspoređuje po hardveru
- Kontekst je ograničen na 8192 tokena (pitanje + odgovor zajedno)

# Zajednička osnova

## 3/3 – generate\_response: jezgra generiranja

```
gen_config = deepcopy(pipe.model.generation_config)
gen_config.max_new_tokens = max_new_tokens
gen_config.temperature = temperature      # 0.3
gen_config.do_sample = do_sample         # False
gen_config.repetition_penalty = 1.1

response = pipe(final_messages, generation_config=gen_config)

# izvuci samo asistentov sadrzaj
for m in response[0]['generated_text']:
    if m['role'] == 'assistant':
        answer = m['content']; break

# ako je odgovor odsjecen (bez . ! ?)
# -> rezi do zadnjeg interpunkcijskog znaka
return answer, ukupno_vrijeme
```

- Postavlja parametre generiranja na kopiji konfiguracije, bez diranja globalnih postavki
- Deterministički način rada (do\_sample=False) daje ponovljive, usporedive rezultate
- Iz izlaza izdvaja samo odgovor asistenta i, ako je odsječen, reže ga do zadnje potpune rečenice

# 1. Vanilla

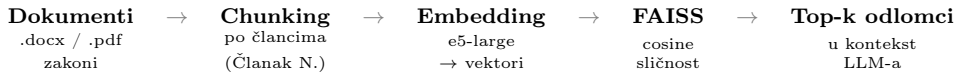
Bez dohvata konteksta

```
def vanilla_chat(user_question):  
    messages = [  
        {"role": "system", "content": SYSTEM_PROMPT},  
        {"role": "user", "content":  
            f"{user_question}\n\nOdgovori NA HRVATSKOM JEZIKU koristeći LATINICU."}  
    ]  
    return generate_response(messages, max_new_tokens=512,  
                             temperature=0.3, do_sample=False)
```

- Bazna linija – šalje samo sistemsku poruku i pitanje, bez ikakvog dohvata
- Model odgovara isključivo iz vlastitog naučenog znanja (rizik halucinacija)

## 2. RAG

### Retrieval-Augmented Generation – pregled



#### Nadogradnja nad zajedničkom osnovom:

- Nove biblioteke: `sentence-transformers` (embeddingi), `faiss-cpu` (pretraga), `python-docx` / `pdfplumber` (čitanje dokumenata)
- Embedding model `intfloat/multilingual-e5-large` [2] – razlikuje "query:" (pitanje) od "passage:" (odlomak)
- `generate_response` dobiva `use_rag=True`: prije generiranja dohvaća odlomke i ubacuje ih u `SYSTEM_PROMPT`



## 2. RAG

### Indeksiranje – rezanje zakona po člancima (chunking)

```
def chunk_by_articles(text, doc_name,
                      max_chunk=1200, min_chunk_len=50):
    article_pattern = r'(Članak\s+\d+[a-z]?\.),'
    parts = re.split(article_pattern, text)

    if len(parts) <= 1:
        return chunk_text_with_overlap(text,
                                       chunk_size=800, overlap=200)

    chunks = []
    for i in range(1, len(parts), 2):
        header = parts[i].strip()
        body   = parts[i+1].strip()
        article_text = f"{header}\n{body}"
        chunks.append(f"[{doc_name}] {article_text}")
    return chunks
```

- Svaki komad = jedan članak zakona, prirodna pravna jedinica
- Ako dokument nema članke → rezanje na komade fiksne duljine s preklapanjem
- Svaki komad nosi oznaku izvornog dokumenta radi slijednosti

## 2. RAG

### Dohvat relevantnih odlomaka

```
def retrieve_relevant_passages(query, top_k=5, score_threshold=0.30):
    fetch_k = min(max(top_k * 5, 50), len(corpus_texts))
    query_embedding = embed_model.encode(["query: " + query])
    faiss.normalize_L2(query_embedding)
    D, I = faiss.index.search(query_embedding, fetch_k)
    candidates = []
    for score, idx in zip(D[0], I[0]):
        if idx == -1 or score < score_threshold:
            continue
        candidates.append({"text": corpus_texts[idx], "score": float(score),
                           "metadata": corpus_metadata[idx]})
    for c in candidates:
        razina = c["metadata"]["razina"]
        c["score"] += max(0, (4 - razina)) * 0.02
    candidates.sort(key=lambda x: x["score"], reverse=True)
    # ... uklanjanje >92% Jaccard duplikata, ograničenje po dokumentu
    return selected[:top_k]
```

- Pitanje se pretvara u vektor i uspoređuje po značenju s odlomcima u FAISS indeksu
- Odlomci ispod praga sličnosti se odbacuju. Viši propisi (Ustav) dobivaju prednost
- Deduplikacija i ograničenja osiguravaju raznolik kontekst iz više zakona

## 2. RAG

### Sastavljanje upita s dohvaćenim kontekstom

```
retrieved = retrieve_relevant_passages(user_query, top_k=rag_top_k)

context_parts = [f"[{i}] {r['text']} (izvor: {r['metadata']['file']})"
                  for i, r in enumerate(retrieved, start=1)]
context_text = "\n\n".join(context_parts)

# ubacuje se u prompt, PRIJE poziva generate_response
final_messages[0]["content"] += f"\n\nKontekst za odgovor:\n{context_text}"

return generate_response(final_messages, max_new_tokens=max_tok,
                          temperature=0.3, do_sample=False)
```

- Dohvaćeni odlomci se formatiraju s brojem i izvorom pa dodaju u poruku
- Time model prelazi s “odgovara po sjećanju” na “odgovara iz priloženih članaka”
- Isti parametri generiranja kao Vanilla – poštena usporedba

### 3. GraphRAG

Graf znanja umjesto ravne liste odlomaka

#### Problem RAG-a

- Ravna lista odlomaka, ne zna da članak upućuje na drugi
- Teško povezuje više zakona (Ustav ↔ Zakon ↔ Pravilnik)
- Kompleksna, međudokumentna pitanja ostaju slabo pokrivena

#### Rješenje GraphRAG-a

- Neo4j graf: čvorovi Zakon → Članak → Stavak
- Veze: UPUCUJE\_NA, ISTA\_TEMA, SPOMINJE (entiteti)
- Dohvat = vektorska pretraga + širenje po vezama grafa

#### Izgradnja grafa (nove komponente):

- Parser čita članke, stavke, upućivanja i klasificira obveze (OBVEZA / ZABRANA / PRAVO)
- BERTić NER model vadi entitete (osobe, organizacije) – zajednički entiteti postaju ISTA\_TEMA veze
- Svaki članak dobiva vektorski embedding pohranjen u Neo4j vektorski indeks

### 3. GraphRAG

Spremanje grafa – veza na Neo4j (AuraDB)

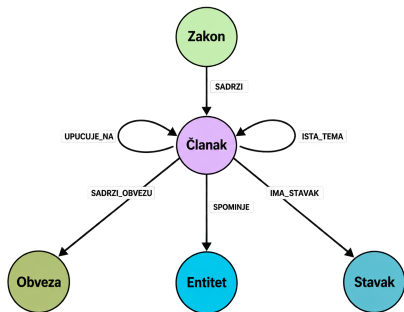
```
neo4j_driver = GraphDatabase.driver(
    NEO4J_URI, auth=(NEO4J_USER, NEO4J_PASSWORD))

def neo4j_run(query, params=None):
    with neo4j_driver.session() as session:
        return list(session.run(query, params or {}))
```

- Graf se pohranjuje u Neo4j instanci (AuraDB) smještenoj u oblaku
- Spajanje ide preko službenog Python klijenta `neo4j.GraphDatabase`
- `neo4j_run` je pomoćna funkcija koja otvara sesiju i izvršava Cypher upit – koriste je sve ostale funkcije za rad s grafom

### 3. GraphRAG

Shema: čvorovi, veze i vektorski indeks



```
CREATE CONSTRAINT FOR (c:Clanak)
  REQUIRE c.id IS UNIQUE

CREATE VECTOR INDEX clanak_embedding
FOR (c:Clanak) ON (c.embedding)
OPTIONS { indexConfig: {
  'vector.dimensions': 1024,    # e5-large
  'vector.similarity_function': 'cosine'
}}
```

```
GRAPH_REL_WEIGHTS = { 'RAZRADJUJE':1.00,
  'ISTA_TEMA':0.95, 'UPUCUJE_NA':0.85,
  'SPOMINJE':0.70 }
```

Slika: Shema grafa u Neo4j  
(CALL db.schema.visualization())

- Constraint jamči jedinstvene članke (nema duplikata, brz dohvat)
- Vektorski indeks omogućuje pretragu po značenju izravno u bazi (bez vanjskog FAISS-a)
- Težine vrsta veza određuju koliko svaka veza utječe pri širenju kroz graf

### 3. GraphRAG

#### Ingestija – upis zakona u graf

```
def graph_ingest_zakon(zakon_data, zakon_id):  
    MERGE (z:Zakon {id: zakon_id})  
    for cl in zakon_data['clanaci']:  
        emb = graph_embed(cl_text) # e5-large  
        # clanak + embedding + veza na zakon  
        MERGE (c:Clanak {id})  
        SET c.tekst, c.embedding = emb  
        MERGE (z)-[:SADRZI]->(c)  
        # stavci, upucivanja, entiteti, obveze:  
        MERGE (c)-[:IMA_STAVAK]->(s)  
        MERGE (src)-[:UPUCUJE_NA]->(t)  
        MERGE (c)-[:SPOMINJE]->(e:Entitet)  
        CREATE (c)-[:SADRZI_OBVEZU]->(o)
```

- Razgrađuje zakon u mrežu čvorova: zakon → članci → stavci, uz upućivanja, entitete i obveze
- Dosljedan MERGE čini upis ponovljivim (bez dupliciranja pri ponovnom pokretanju)
- Rezultat: zakon više nije ravan tekst nego povezana struktura

### 3. GraphRAG

Dohvat: hibridni pristup – obilazak grafa i rangiranje

```
MATCH (seed:Clanak) WHERE seed.id IN $ids
OPTIONAL MATCH (seed)-[r:UPUCUJE_NA|RAZRADJUJE|ISTA_TEMA]-(n1:Clanak)
OPTIONAL MATCH (seed)-[:SPOMINJE]->(e:Entitet)<-[:SPOMINJE]-(n2:Clanak)
WHERE n2.zakon_id <> seed.zakon_id
RETURN seed.id AS sid,
        collect(DISTINCT {id: n1.id, tekst: n1.tekst, rel: type(r)})
+ collect(DISTINCT {id: n2.id, tekst: n2.tekst, rel: 'SPOMINJE'})
AS susjedi
```

- Od pronađenih članaka širi pretragu na povezane – na dva načina
  - Strukturno: izravne veze (upućivanje, razrada, ista tema)
  - Tematski: zajednički entitet
- Tematsko povezivanje ograničeno na različite zakone – hvata međudokumentne odnose



### 3. GraphRAG

Orkestracija: od upita do odgovora

```
def graph_retrieve_context(query, kategorija=None):
    top_k = GRAPH_TOP_K.get(kategorija, GRAPH_TOP_K_DEFAULT)
    query_emb = embed_model.encode(["query: " + query])[0].tolist()

    seeds = vector_search(query_emb, top_k)           # queryNodes, clanak+stavak
    prosireno = obidji_susjede(seeds)                 # Cypher upit (prosli slajd)
    rangirano = rangiraj_i_dedupliciraj(prosireno)     # tezina x GRAPH_REL_WEIGHTS
    return sastavi_kontekst(rangirano[:top_k])

def graph_rag_chat(user_question, kategorija=None):
    context = graph_retrieve_context(user_question, kategorija=kategorija)
    messages = [
        {"role": "system", "content": SYSTEM_PROMPT + f"\n\nKontekst za odgovor:\n{context}"},
        {"role": "user", "content": f"{user_question}\n\nOdgovori NA HRVATSKOM JEZIKU koristeći LATINICU."}
    ]
    return generate_response(messages, max_new_tokens=512,
                             temperature=0.3, do_sample=False)
```

- Dohvat u četiri koraka: vektorska pretraga → širenje grafom → rangiranje → sastavljanje konteksta
- top\_k se skalira prema složenosti pitanja (5 / 8 / 12)

# Usporedba

Što svaki sloj dodaje

## Vanilla

- Izvor: samo naučeno znanje
- Dohvat: nema
- Rizik: halucinacije članaka
- Najbrži, najslabija točnost
- Uloga: bazna linija

## RAG

- Izvor: FAISS vektorski indeks
- Dohvat: top-k odlomaka
- Chunk = članak zakona
- Navodi izvore
- Slabije kod povezivanja zakona

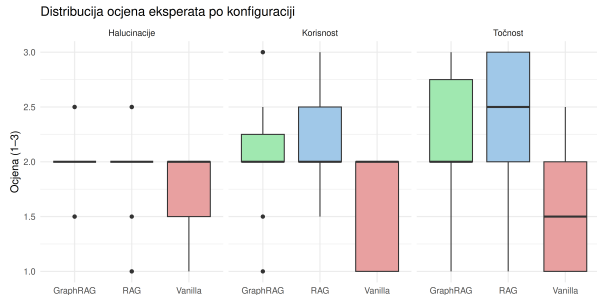
## GraphRAG

- Izvor: Neo4j graf znanja
- Dohvat: vektor + veze grafa
- Hvata upućivanja i teme
- Najsloženiji za izgradnju

# Statistička analiza

## Glavni rezultati

| Konfig.  | Dimenzija    | $\bar{x}$ |
|----------|--------------|-----------|
| Vanilla  | Točnost      | 1,56      |
|          | Korisnost    | 1,65      |
|          | Halucinacije | 1,69      |
| RAG      | Točnost      | 2,32      |
|          | Korisnost    | 2,13      |
|          | Halucinacije | 2,02      |
| GraphRAG | Točnost      | 2,27      |
|          | Korisnost    | 2,11      |
|          | Halucinacije | 2,03      |



**RAG i GraphRAG statistički značajno bolji od Vanilla.**

**RAG naspram GraphRAG – bez značajne razlike ni na jednoj mjeri.**

- Statistička analiza pokazala je da između tri konfiguracije postoje **statistički značajne razlike** na svim promatranim mjerama
- Naknadne parne usporedbe: **RAG** i **GraphRAG** značajno bolji od **Vanilla** pristupa, uz velike veličine učinka, dok razlika između RAG-a i GraphRAG-a nije statistički značajna
- Analiza međuocjenjivačkog slaganja pokazala je **nisku pouzdanost** ljudskih procjena, zbog čega rezultate ljudske evaluacije treba interpretirati oprezno
- Korelacijska analiza pokazala je umjerene do visoke povezanosti nekih automatskih (kosinusna sličnost) i LLM baziranih metrika s ljudskim procjenama

# Iterativni razvoj

## Deset iteracija: ključne pouke

- **Curenje podataka:** v3 postigla iznenađujućih 90 % (korišten LLM-as-a-Judge kao ocjenjivač) točnosti jer je evaluacijski skup pitanja slučajno bio uključen u korpus
- **Jezično miješanje:** v6/v9 odgovori klizili u bugarski/ćirilicu; agresivne prompt zabrane problem **pogoršale**
- **Model i parametri:** **EuroLLM-22B** odabran nad Qwen2.5-14B (bolja tokenizacija za hrvatski); `repetition_penalty` kalibriran 0,6 → 1,3 → konačno **1,1**
- **Zašto GraphRAG nije briljirao:** rijedak graf (538 članaka, svega 7 međudokumentnih veza) i obilazak ograničen na 1 korak [3], [4]. Mali broj pitanja koja zahtijevaju međudokumentne veze (npr. Ustav → Zakon → Pravilnik) – evaluacijski skup nije dovoljno velik da pokaže prednost GraphRAG-a

## Pouka

Separacija evaluacijskog skupa od korpusa, oprez s prompt engineeringom i kalibrirani parametri generiranja ključni su za pouzdanost eksperimenta.

# Zaključak

## Glavni nalazi i budući rad

- Zajednička osnova (isti model, kvantizacija, generacija) drži usporedbu poštenom – mijenja se samo dohvat konteksta
- Dohvat konteksta (**RAG**, **GraphRAG**) statistički značajno nadmašuje **Vanilla** na svim promatranim mjerama
- **GraphRAG ne donosi mjerljivu prednost** nad RAG-om – izostanak dokaza razlike, ne dokaz jednakosti. Ipak, jedini eksplicitno modelira međudokumentne odnose pravnih propisa
- Vrijednost rada i u dokumentiranju iterativnog razvoja (curenje podataka, jezično miješanje, kalibracija parametara)
- **Budući rad:** dublji, višeskočni obilazak grafa. Hibridni sustav koji dinamički bira RAG ili GraphRAG. Veći evaluacijski skup.

Vanilla (znanje)  $\rightarrow$  RAG (dohvat)  $\rightarrow$  GraphRAG (dohvat + struktura)

- [1] T. Dettmers, A. Pagnoni, A. Holtzman i L. Zettlemoyer, „Qlora: Efficient finetuning of quantized llms,” *Advances in Neural Information Processing Systems*, sv. 36, str. 10 088–10 115, 2023.
- [2] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder i F. Wei, „Multilingual E5 Text Embeddings: A Technical Report,” *arXiv preprint arXiv:2402.05672*, 2024.
- [3] Z. Xiang, C. Wu, Q. Zhang i dr., „When to use graphs in rag: A comprehensive analysis for graph retrieval-augmented generation,” *International Conference on Learning Representations*, sv. 2026, 2026., str. 66 145–66 178.
- [4] B. Peng, Y. Zhu, Y. Liu i dr., „Graph retrieval-augmented generation: A survey,” *ACM Transactions on Information Systems*, sv. 44, br. 2, str. 1–52, 2025.





Zahvaljujem na pažnji!  
**Pitanja?**